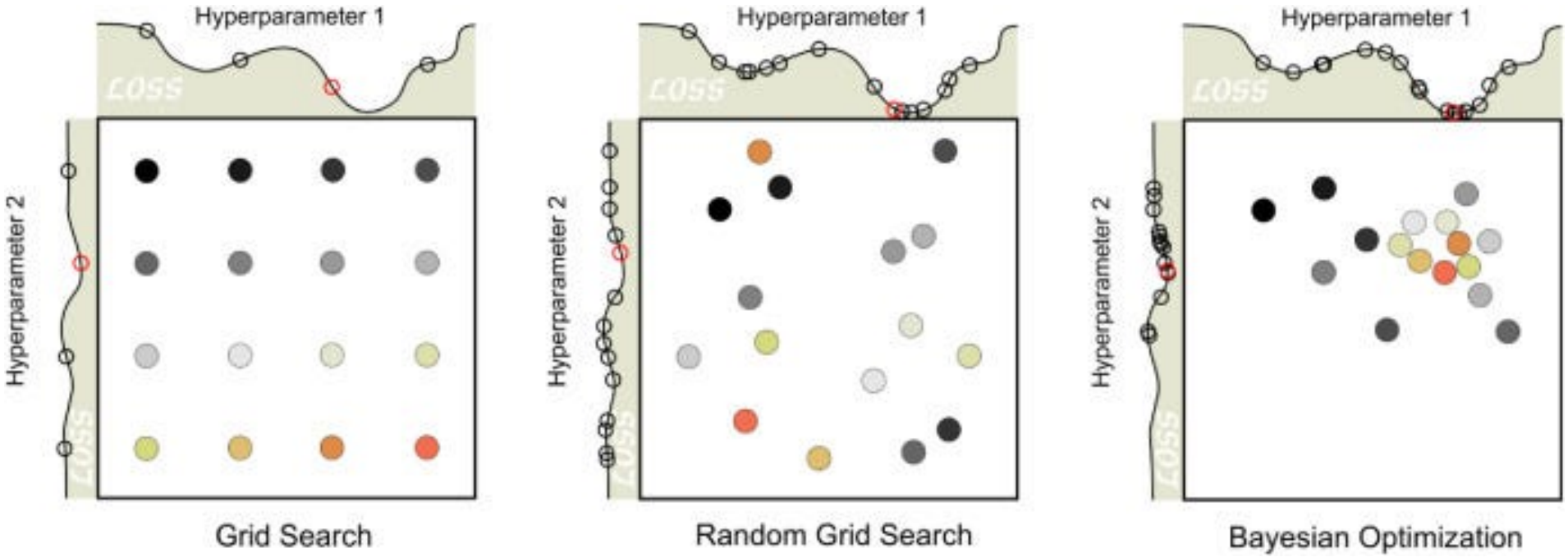
AutoML, Autoresearch, MLOps +@

DOE는 Research Agent의 Harness가 될 수 있는가?

배경 정리 + harness 제안

1. AutoML이란 무엇인가?

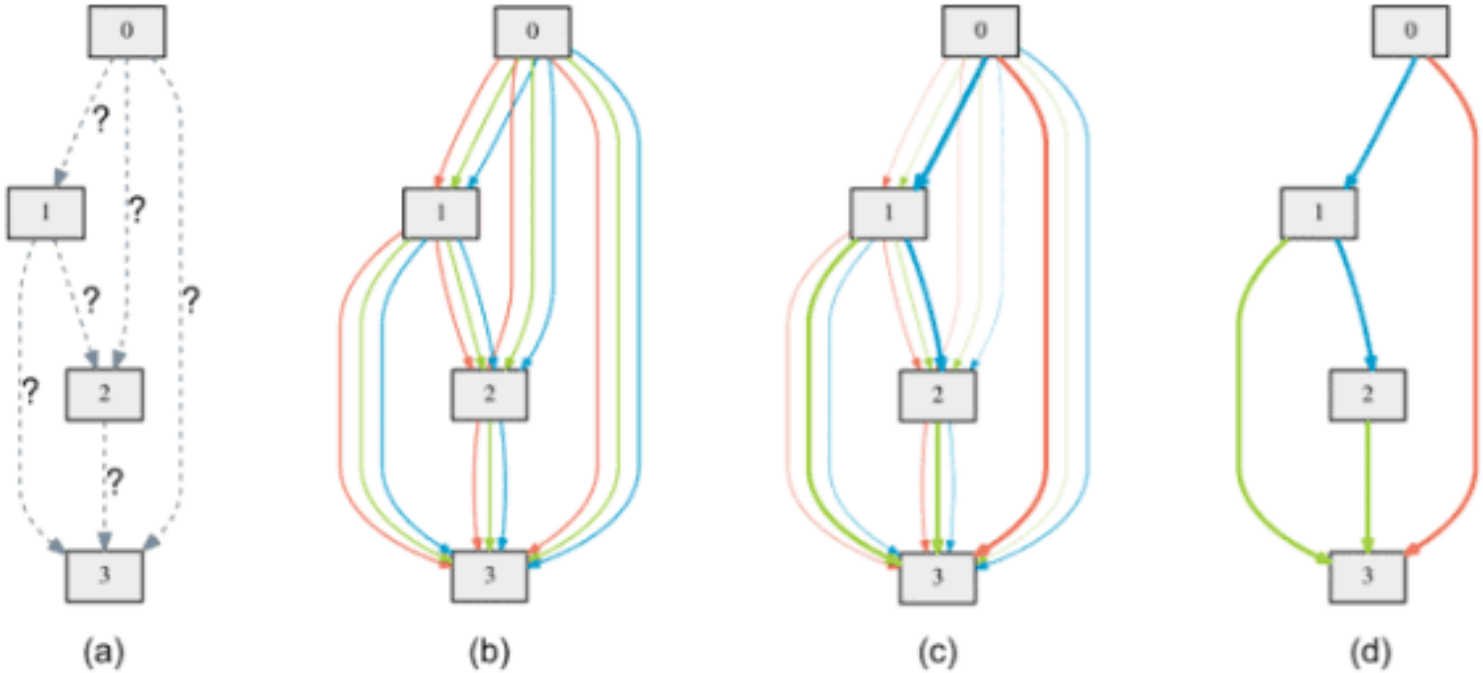
- 모델 개발 탐색 일부 자동화
- 대표 대상: `model selection`, `hyperparameter tuning`, `pipeline search`
- 핵심: 비교적 주어진 `search space` 안 최적 설정 탐색



출처 이미지

2. `Neural Architecture Search`는 AutoML의 확장이다

- parameter 대신 architecture 탐색
- AutoML의 `더 넓은 search space` 확장선
- 그래도 중심은 여전히 모델/파이프라인 후보 탐색



3. `Autoresearch`는 어떻게 등장했고 무엇이 다른가?

- karpathy/autoresearch: 작은 training setup 위 `read → edit → run → keep-or-revert` loop 제시
- 연구 workflow 일부를 agent가 직접 수행하며, 설정 탐색을 넘어 `code`, `module`, `experiment` 자체 수정
- AutoML의 `fixed search space` 바깥으로 확장
- 이후 RD-Agent, AI-Scientist, GPT Researcher 등으로 빠르게 확장

4. Agent 작업 흐름

- 코드 읽기, baseline 파악
- 작은 가설 하나 선택
- 학습 코드나 설정 수정
- 짧은 실험 실행, metric 확인
- 나쁘면 revert, 의미 있으면 keep
- 핵심: `edit 한 번`이 아니라 `짧은 실험 loop의 누적`

```
Question → Read → Edit → Run → Analyze → Next experiment
```

5. AutoML vs. Autoresearch

항목	AutoML	Autoresearch
탐색 대상	config, pipeline, architecture	hypothesis, code, module, experiment
핵심 질문	어떤 설정이 가장 좋은가	다음에 어떤 실험을 해야 하는가
edit 단위	parameter / architecture	code / module / pipeline / experiment
평가 방식	objective 중심	objective + reasoning + iteration
위험	비효율적 탐색	incoherent search, metric hacking
필요한 인프라	experiment infra	experiment + memory + harness

6. 사용례와 확장

사용례

- 문헌 조사 / deep research: GPT Researcher
- 코드 수정 + 실험 반복: karpathy/autoresearch, RD-Agent
- end-to-end 연구 자동화: AI-Scientist

확장

- benchmark / evaluation: MLE-bench, MLAgentBench, MLR-Bench
- plugin / skill 생태계: awesome-autoresearch, Awesome Auto Research Tools
- memory, reusable modules, hardware fork

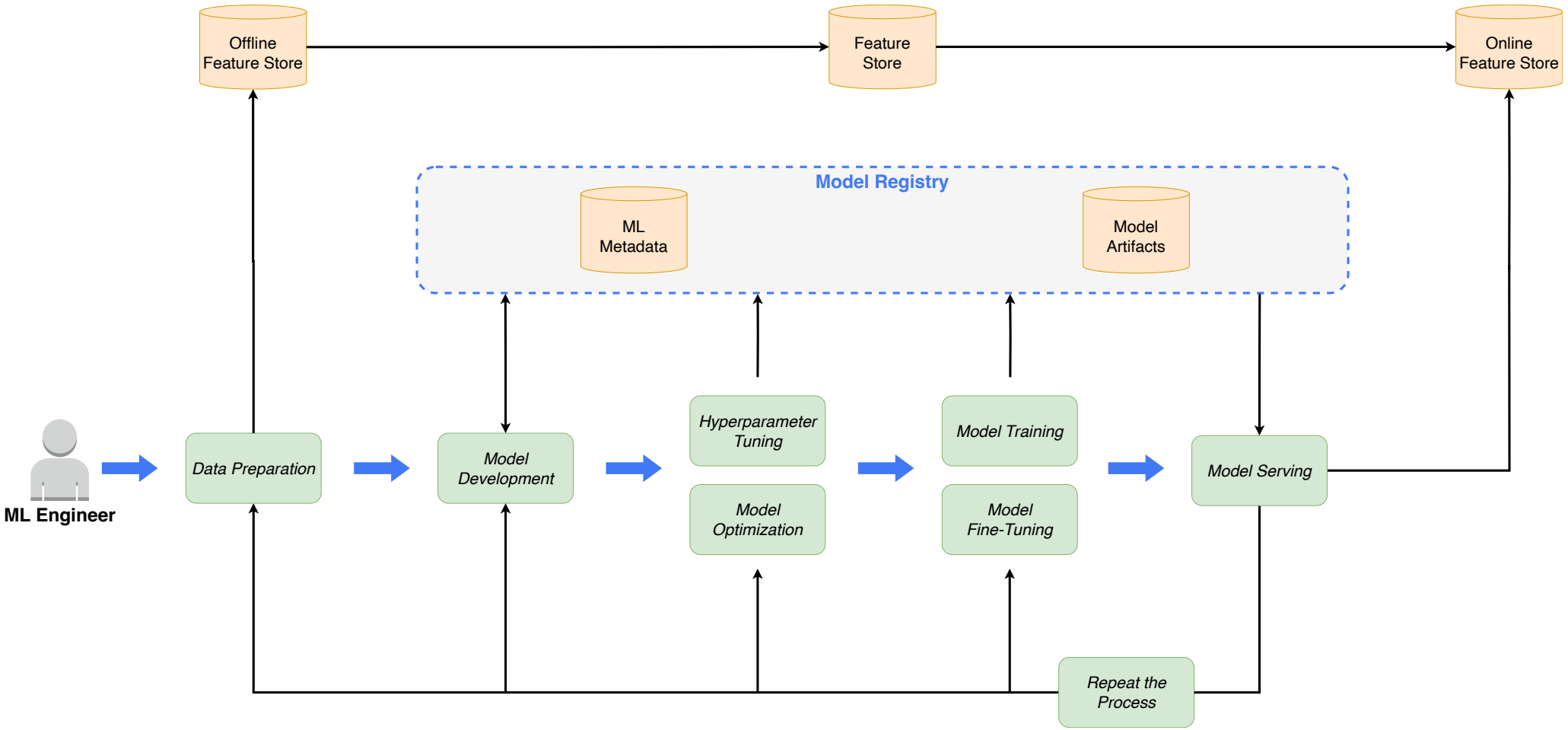
7. 체계적인 실험 관리의 필요

- 공통 문제: `많은 run` 비교와 누적
- 필수 요소: `tracking`, `lineage`, `orchestration`
- agent edit가 들어오면 `artifact`, `promotion`, `monitoring`, `cost control` 중요도 상승
- 결국 운영 문제

8. AutoML과 Autoresearch가 공통으로 요구하는 MLOps 요소

요소	AutoML에서의 역할	Autoresearch에서의 역할
tracking	sweep 비교	hypothesis / code edit history 비교
orchestration	search job 실행	agent + eval job 실행
registry / lineage	best model 승격	experiment / prompt / code provenance 보존
monitoring / cost	retrain trigger, SLO	budget, drift, unsafe promotion guardrail

9. MLOps는 모델 개발, 관리, 배포 파이프라인을 유지 관리하는 작업이다



- Autoresearch loop는 이 큰 ML lifecycle 안의 일부

10. Autoresearch는 어디서 흔들리나

- 실험이 즉흥적으로 이어지기 쉬움
- 왜 이 실험을 했는지 attribution이 약함
- 큰 수정, 작은 튜닝, 검증 실험이 섞이기 쉬움
- robustness, replication, interaction 확인이 뒤로 밀림
- 잘 정리된 random search로 퇴화할 위험

11. 어떤 Harness가 필요한가

- 무엇을 먼저 볼지 정하는 우선순위
- 어떤 조합을 함께 볼지 정하는 규율
- 탐색 단계와 검증 단계 분리
- 작은 수정과 큰 수정을 다르게 다루는 운영 규칙
- 실패도 정보로 남기는 구조
- 다음 라운드를 설계하는 순차 실험 체계

12. DoE가 그 Harness가 될 수 있을까

autoresearch need	DoE concept
무엇부터 실험할까	screening
어떤 조합을 함께 볼까	factorial design
언제 정밀 탐색할까	sequential design
안정적인가	robust design
비율은 어떻게 나눌까	mixture / allocation

13. DoE에서 빌려오는 개념

- `screening` : 중요한 요인부터 좁히기
- `factorial thinking` : interaction 보기
- `sequential design` : 라운드별 정교화
- `robust design` : 평균이 아니라 안정성까지 확인
- `mixture / allocation` : 예산과 비율 배분

14. Vanilla Agent와 DoE-Guided Agent

Vanilla autoresearch	DoE-guided autoresearch
edit → run → score 반복	factor 정의 → screening → interaction → refinement → robustness
실험 성격 혼재	실험 타입 분리
best score 중심	effect / interaction / stability 축적
장기 구조 약함	라운드 기반 운영

15. 비교한 Agents

Agent	설명
Vanilla Agent	기본 autoresearch loop
DoE Agent	DoE 기반 실험 설계와 단계화
DoE + X Agent	DoE + 추가 전략 모듈

- 비교 포인트: 성능만이 아니라 탐색의 질과 실험 구조

16. 실험 설정

항목	설정
Task	TBD
Budget	TBD
Metric	TBD
Common base model	TBD
Common runtime rule	TBD
Agent-specific difference	TBD

- 라운드 예시: screening → interaction / module refinement → local tuning → robustness

17. 결과: 성능, 효율, 안정성

실험 후 채움

Agent	Best Score	Avg Gain	Runs Used	Cost	Robustness Pass
Vanilla	TBD	TBD	TBD	TBD	TBD
DoE	TBD	TBD	TBD	TBD	TBD
DoE + X	TBD	TBD	TBD	TBD	TBD

18. 결과: 탐색 궤적

- `실험 후 채움`
- `x-axis` : run budget
- `y-axis` : best-so-far metric
- 비교 포인트: 수렴 속도, run 효율, plateau 형태
- 추가 후보: 실패 유형 분포 / `Tic-Tac-Toe` 비율 / robustness pass rate

19. 왜 DoE가 도움됐나

- 실험 우선순위 구조화
- 무의미한 탐색 폭 축소
- interaction을 더 의식적으로 다룸
- tuning과 validation 분리
- robustness를 과정 안에 포함
- 결과가 다음 라운드 지식으로 축적

20. 한계

- 문헌 조사나 가설 생성 자체를 대체하진 못함
- 완전히 open-ended한 구조 탐색은 factorization이 어려움
- classical DoE는 코드 중심 search space에 바로 맞지 않을 수 있음
- large-scale 학습에서는 replication 비용이 큼
- 평가 함수가 빈약하면 DoE도 잘못된 목표를 최적화할 수 있음

21. 결론

- autoresearch는 유망하지만 실험 규율이 약함
- DoE는 autoresearch의 experimentation harness 후보
- 앞으로의 과제는 더 좋은 agent뿐 아니라 더 좋은 harness

> 더 큰 agent보다 더 강한 experimental harness

23. References

구분	예시
curated landscape	awesome-autoresearch, Awesome Auto Research Tools
end-to-end systems	karpathy/autoresearch, RD-Agent, AI-Scientist
deep research	GPT Researcher
evaluation	MLE-bench, MLAgentBench, MLR-Bench
visuals	AutoML image, NAS image, Kubeflow model registry lifecycle image